

Ansh Thapa

B.Tech CSE

CU24250224

Experiment No. 3

Implementation and Analysis of Spatial Filtering Techniques using Low-Pass and High-Pass Filters

Aim

To implement and analyze spatial domain filtering techniques using low-pass and high-pass filters for image smoothing, noise reduction, edge enhancement, and feature preservation using Python and OpenCV

Purpose

Spatial filtering is one of the fundamental operations in digital image processing, used to enhance image quality by modifying pixel values based on their neighboring pixels. Low-pass filters are primarily used to reduce noise and smooth images, whereas high-pass filters emphasize edges, fine details, and abrupt intensity changes, which are essential for feature extraction and object recognition.

In this experiment, students will implement various spatial filtering techniques on real-world images and evaluate their impact on image quality, edge preservation, and noise removal. The experiment demonstrates the significance of filtering as a preprocessing step in computer vision applications such as medical imaging, surveillance, remote sensing, and autonomous systems.

Prerequisites

- Basic knowledge of digital image processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), NumPy, and Matplotlib
- Understanding of image kernels and convolution operations
- Jupyter Notebook, Google Colab, or Visual Studio Code

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
from google.colab import files
uploaded = files.upload()

filename = list(uploaded.keys())[0]
img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)
```

[Choose Files](#) 2025_Nitro...40x2400.jpg
2025_Nitro_Default_3840x2400.jpg(image/jpeg) - 3420968 bytes, last modified: 11/18/2024 - 100% done
Saving 2025_Nitro_Default_3840x2400.jpg to 2025_Nitro_Default_3840x2400 (1).jpg

```
plt.imshow(img, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.show()

print("Shape:", img.shape)
print("Min/Max/Mean pixel value:", img.min(), img.max(), img.mean())
print("Std Deviation (contrast measure):", img.std())
```

Original Image



Shape: (2400, 3840)
Min/Max/Mean pixel value: 0 255 39.16507476128472
Std Deviation (contrast measure): 45.99699600271422

```
gaussian_blur = cv2.GaussianBlur(img, (5, 5), sigmaX=1.5)

plt.imshow(gaussian_blur, cmap='gray')
plt.title("Gaussian Blur")
plt.axis('off')
plt.show()
```

Gaussian Blur



```
median_filtered = cv2.medianBlur(img, 5)

plt.imshow(median_filtered, cmap='gray')
plt.title("Median Filter")
plt.axis('off')
plt.show()
```

Median Filter



```
average_filtered = cv2.blur(img, (5, 5))

fig, ax = plt.subplots(1, 2, figsize=(10,5))
ax[0].imshow(gaussian_blur, cmap='gray'); ax[0].set_title("Gaussian Blur"); ax[0].axis('off')
```

```
ax[1].imshow(average_filtered, cmap='gray'); ax[1].set_title("Average Filter"); ax[1].axis('off')
plt.show()
```

Gaussian Blur



Average Filter



```
laplacian = cv2.Laplacian(img, cv2.CV_64F, ksize=3)
laplacian_abs = cv2.convertScaleAbs(laplacian)
```

```
plt.imshow(laplacian_abs, cmap='gray')
plt.title("Laplacian Filter")
plt.axis('off')
plt.show()
```

Laplacian Filter



```
sobel_x = cv2.Sobel(img, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_64F, 0, 1, ksize=3)
```

```
sobel_x_abs = cv2.convertScaleAbs(sobel_x)
sobel_y_abs = cv2.convertScaleAbs(sobel_y)
sobel_combined = cv2.convertScaleAbs(np.sqrt(sobel_x**2 + sobel_y**2))
```

```
fig, ax = plt.subplots(1, 3, figsize=(15,5))
ax[0].imshow(sobel_x_abs, cmap='gray'); ax[0].set_title("Sobel X"); ax[0].axis('off')
ax[1].imshow(sobel_y_abs, cmap='gray'); ax[1].set_title("Sobel Y"); ax[1].axis('off')
ax[2].imshow(sobel_combined, cmap='gray'); ax[2].set_title("Sobel Combined"); ax[2].axis('off')
plt.show()
```

Sobel X



Sobel Y



Sobel Combined



```
import time
```

```
def timeit(func, *args, **kwargs):
    t0 = time.time()
    result = func(*args, **kwargs)
    return result, time.time() - t0
```

```
_, t_gauss = timeit(cv2.GaussianBlur, img, (5,5), 1.5)
```

```
_, t_median = timeit(cv2.medianBlur, img, 5)
_, t_avg = timeit(cv2.blur, img, (5,5))
_, t_lap = timeit(cv2.Laplacian, img, cv2.CV_64F)
_, t_sobel = timeit(cv2.Sobel, img, cv2.CV_64F, 1, 0)

print(f"Gaussian: {t_gauss:.5f}s")
print(f"Median : {t_median:.5f}s")
print(f"Average : {t_avg:.5f}s")
print(f"Laplacian: {t_lap:.5f}s")
print(f"Sobel : {t_sobel:.5f}s")

print("Std dev original patch:", img[10:50,10:50].std())
print("Std dev gaussian patch:", gaussian_blur[10:50,10:50].std())
print("Std dev average patch :", average_filtered[10:50,10:50].std())
```

```
Gaussian: 0.01877s
Median : 0.02458s
Average : 0.00550s
Laplacian: 0.03299s
Sobel : 0.03499s
Std dev original patch: 0.04993746088859545
Std dev gaussian patch: 0.02499218627891526
Std dev average patch : 0.0
```

```
images = [img, gaussian_blur, median_filtered, average_filtered, laplacian_abs, sobel_combined]
titles = ["Original", "Gaussian Blur", "Median Filter", "Average Filter", "Laplacian", "Sobel Combined"]

fig, axes = plt.subplots(2, 3, figsize=(15,10))
for ax, im, title in zip(axes.ravel(), images, titles):
    ax.imshow(im, cmap='gray')
    ax.set_title(title)
    ax.axis('off')
plt.tight_layout()
plt.show()
```



Interpretation

- Low-pass filters (Gaussian, Median, Average) reduce noise but also blur some detail/edges.
- Gaussian → uses weighted averaging, so it preserves edges better
- Average → gives equal weight to all pixels, causing more blur
- Median → most effective for salt-and-pepper noise, keeps edges relatively sharp

- High-pass filters (Laplacian, Sobel) highlight edges/fine details but can also amplify noise.
- Laplacian → a second-order derivative operator, sensitive to edges in all directions (and to noise)
- Sobel → a first-order derivative operator, gives directional gradients (X/Y), slightly more robust to noise than Laplacian
- Trade-off: More smoothing means more detail loss; more edge enhancement means more risk of noise amplification. That's why practical systems often combine both — denoise first (Gaussian), then detect edges (Sobel).

Questions!

1. What is spatial filtering? How is it used in digital image processing?

Spatial filtering is a technique where operations are performed directly on the pixels of an image (as opposed to frequency domain filtering, where the image is first transformed). A small kernel/mask slides over the image, and each pixel's value is modified based on the values of its neighboring pixels. It is used in image processing for noise reduction, smoothing, sharpening, and edge detection.

2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples. Low-Pass Filters (LPF)

Allow low-frequency components (smooth, gradual regions) to pass and suppress high-frequency components (noise, sharp edges). Work by averaging neighboring pixels, which smooths out sudden intensity changes. Effect: blurs/smooths the image. Examples: Average Filter, Gaussian Filter, Median Filter Applications: noise reduction, pre-processing before edge detection

High-Pass Filters (HPF)

Allow high-frequency components (edges, fine details) to pass and suppress low-frequency components (flat/smooth regions). Work by calculating the difference/rate of change (derivative) between neighboring pixels, highlighting areas of sudden intensity change. Effect: sharpens edges and details; flat areas become dark/zero. Examples: Sobel Operator, Laplacian Operator, Prewitt Operator Applications: edge detection, image sharpening, feature extraction

Q3. Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications.

Average Filter: Replaces each pixel with the simple arithmetic mean of all pixels in its neighborhood, giving equal weight to all of them. It's a linear filter, best for general smoothing, but blurs edges heavily.

Gaussian Filter: Also averaging-based, but uses a weighted average where the center pixel gets the highest weight, decreasing outward following a Gaussian (bell-curve) distribution. It's linear, preserves edges better than the average filter, and is widely used for reducing sensor/Gaussian noise.

Median Filter: Sorts all pixel values in the neighborhood and picks the middle (median) value as the output. It's a non-linear filter, extremely effective for removing salt-and-pepper (impulse) noise, and keeps edges sharper than the other two.

Q4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

Salt-and-pepper noise creates extreme pixel values (0 or 255) very different from the actual surrounding pixels. When the median filter sorts the neighborhood values, these extreme outliers end up at the very beginning or end of the sorted list — so they get excluded when the middle (median) value is picked. In contrast, Average and Gaussian filters include these extreme values in their calculation, which just smears the noise across nearby pixels instead of removing it. This is why the Median filter removes impulse noise much more effectively while keeping edges sharp.

Q5. Explain the role of convolution kernels in spatial filtering.

A convolution kernel (or mask) is a small matrix (e.g., 3×3 or 5×5) that slides over the image pixel by pixel. At each position, the kernel's values are multiplied with the corresponding image pixel values and summed up — this sum becomes the new value of the output pixel at that position. The values inside the kernel determine what effect the filter produces: an averaging kernel produces smoothing, while a derivative-based kernel (like Sobel or Laplacian) produces edge detection. Essentially, the kernel defines the mathematical rule that transforms the image.

Q6. What is the purpose of the Sobel and Laplacian operators in edge detection?

Sobel Operator: A first-order derivative operator that calculates the gradient (rate of change) of intensity separately in the X and Y directions. This gives directional edge information — horizontal and vertical edges can be detected separately, and combined for a full edge map.

Laplacian Operator: A second-order derivative operator that detects edges in all directions simultaneously (direction-independent). It identifies edges through zero-crossings and also highlights fine details/texture.

Both are used to locate sharp intensity transitions (edges) in an image, which usually correspond to object boundaries.

Q7. Why are filtering operations considered an essential preprocessing step in computer vision?

Real-world images are often noisy, low-contrast, or blurred due to camera sensor limitations, lighting variations, or compression artifacts. Filtering operations:

Clean up noise and improve image quality Enhance important features like edges and boundaries Improve the accuracy of downstream tasks such as object detection, segmentation, and recognition

Without proper preprocessing, computer vision algorithms may mistake noise for actual features, leading to unreliable results. That's why filtering is almost always the first/early step in any vision pipeline.

Q8. Discuss the trade-off between image smoothing and edge preservation during filtering.

Smoothing filters (like Gaussian, Average) reduce noise by increasing kernel size — but the more smoothing applied, the more edges also get blurred, since edges are also a form of "high-frequency" information that smoothing filters attenuate.

On the other hand, if edges are preserved sharply (less smoothing), noise won't be properly removed.

So there's always a balance to strike:

Small kernel size → edges preserved, but some noise remains Large kernel size → noise well removed, but edges get blurred

Advanced techniques like Bilateral Filtering or Edge-Preserving Smoothing handle this trade-off better — they reduce noise while intelligently preserving edges.

Q9. Mention four real-world applications where spatial filtering techniques are widely used.

1. Medical Imaging – denoising MRI/CT scans for clearer diagnosis
2. Autonomous Vehicles – lane detection and obstacle boundary detection
3. Industrial Quality Inspection – detecting defects/cracks in manufactured products
4. Photography/Mobile Camera Apps – noise reduction in low-light photos and image sharpening

Q10. Compare spatial domain filtering with frequency domain filtering in terms of implementation and practical applications.

Spatial Domain Filtering works directly on the pixels of the image. A small kernel (like 3×3 or 5×5) slides over the image, and each pixel's value is calculated based on its neighboring pixels through convolution. This method is simple, intuitive, and easy to implement. Examples include Gaussian Blur, Median Filter, Average Filter, Sobel, and Laplacian. It's fast for small kernel sizes and is commonly used for real-time applications like noise reduction and edge detection.

Frequency Domain Filtering works differently — it first converts the image from the spatial domain into the frequency domain using a mathematical transform called the Fourier Transform. In this domain, the image is represented in terms of frequencies rather than pixel positions. Filtering is then applied by removing or keeping certain frequency ranges (like removing high frequencies for smoothing, or low frequencies for sharpening). After filtering, an Inverse Fourier Transform is applied to convert the image back to the spatial domain. Examples include Ideal Low-pass/High-pass filters, Butterworth filters, and Notch filters.